

Electrical and Computer Engineering ECE3622 Embedded Systems Design



Introduction to Counter/Timers

Dennis Silage, PhD
silage@temple.edu

Counter/timer hardware is a crucial component of most embedded systems. In some cases a timer is needed to measure elapsed time; in others we want to count or time some external events.

Counter/timer hardware is a crucial component of most embedded systems. In some cases, a timer measures elapsed time (counting processor clock ticks). In others, we want to count or time external events. The names counter and timer can be used interchangeably when talking about the hardware. The difference in terminology has more to do with how the hardware is used in each application.

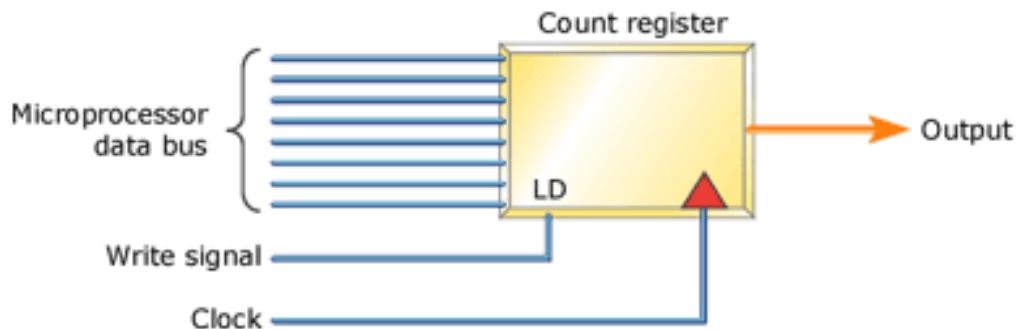


Figure 1: A simple counter/timer

Figure 1 shows a simple timer similar to those often included on-chip within a microcontroller. You could build something similar from a couple of 74HC161 counters or a programmable logic device. The timer shown consists of a loadable 8-bit count register, an input clock signal, and an output signal. Software loads the count register with an initial value between 0x00 and 0xFF. Each subsequent transition of the input clock signal increments that value.

When the 8-bit count overflows, the output signal is asserted. The output signal may thereby trigger an interrupt at the processor or set a bit that the processor can read. To restart the timer, software reloads the count register with the same or a different initial value.

If a counter is an up counter, it counts up from the initial value toward 0xFF. A down counter counts down, toward 0x00.

A typical counter will have some means to start the counter running once it is loaded, usually by setting a bit in a control register. This is not shown in the figure. A real counter would generally also provide a way for the processor to read the current value of the count register at any time, over the data bus.

Semi-Automatic

A timer with automatic reload capability will have a latch register to hold the count written by the processor. When the processor writes to the latch, the count register is written as well. When the timer later overflows, it first generates an output signal. Then, it automatically reloads the contents of the latch into the count register. Since the latch still holds the value written by the processor, the counter will begin counting again from the same initial value.

Such a timer will produce a regular output with the same accuracy as the input clock. This output could be used to generate a periodic interrupt like a real-time operating system (RTOS) timer tick, provide a baud rate clock to a UART, or drive any device that requires a regular pulse.

A variation of this feature found in some timers uses the value written by the processor as the endpoint rather than the initial count. In this case, the processor writes into a terminal count register that is constantly compared with the value in the count register. The count register is always reset to zero and counts up. When it equals the value in the terminal count register, the output signal is asserted. Then the count register is reset to zero and the process repeats. The terminal count remains the same. The overall effect is the same as an overflow counter. A periodic signal of a pre-determined length will then be produced.

If a timer supports automatic reloading, it will often make this a software-selectable feature. To distinguish between a count that will not repeat automatically and one that will, the hardware is said to be in one of two modes: one-shot or periodic. The mode is generally controlled by a field in the timer's control register.

Input Capture

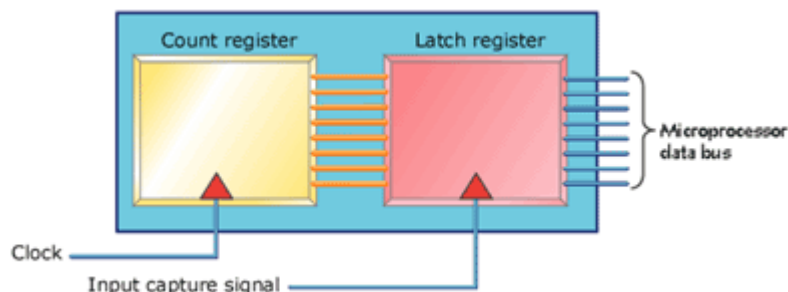


Figure 2: An input capture timer

An input capture timer, like the one shown in Figure 2, has a latch connected to the timer's count register. The timer is run at a constant clock rate (usually a derivative of the processor clock), so that the count register is constantly incrementing (or decrementing, for a down counter). An external signal latches the value of the free-running timer into the processor-visible register and generates an output signal (typically an interrupt).

One use for an input capture timer is to measure the time between the leading edge of two pulses. By reading the value in the latch and comparing it with a previous reading, the software can determine how many clock cycles elapsed. In some cases, the timer's count register might be automatically reset just after its value is latched. If so, the software can directly interpret the value it reads as the number of clock ticks elapsed. An input capture pin can usually be programmed to capture on either the rising or falling edge of the input signal.

Options Abound

Many timers provide a means to prescale the input clock signal. For example, the 8-bit timer in Atmel's AT90S8515 microcontroller can be incremented with every processor clock cycle, every 8th, every 64th, every 256th, or every 1,024th. Selecting a less frequent update cycle is called prescaling the input clock. Similarly, each increment could occur on either the rising or falling edge of some other signal entirely. In the Atmel part, these features are software-selectable.

Some timers can directly control a general-purpose I/O pin. When an overflow occurs, the pin can be automatically set to 1, reset to 0, or toggled. This can be useful in, for example, generating a PWM signal.¹ Using two different initial or terminal count values and a one-shot timer that toggles the I/O pin on overflow, the pin could be set to 1 for a desired amount of time, then 0 for a different amount of time, then 1 again, and so on. The period of the PWM signal would be a function of the sum of the two timer lengths. The duty cycle would then be the length of time that the pin is set to 1 as a percentage of the period.

Although the examples here have focused on 8-bit timers, the concepts apply to larger timers as well.